

A background image of a field of colorful flowers, including pink, purple, and yellow blooms, under a clear blue sky. The flowers are in various stages of bloom, and the overall scene is bright and natural.

AI-Assisted Product Prototyping

Dr. Kornelia Papp

Agenda

01

Bartosz Nadworny (AI Staff Engineer): Engineering with AI

02

Vibe coding

03

David Oort Alonso (CEO): Introduction to Bloom

04

Build & Experiment

Session goals

In this session, we explore how vibe coding turns high-level creative intent into working mobile software.

What is Vibe Coding?

Understand how natural language, creative intent, and AI-assisted development can be used to design and build functional native mobile applications.

How does the developer role change?

Analyze the shift from programmer to architect, where engineers increasingly define systems, constraints, interfaces, and product intent rather than writing every line of code.

How can Bloom support solution creation?

Understand how natural language, creative intent and AI-assisted development can be used to design and build functional applications for mobile and web.

Agenda

01

Bartosz Nadworny (AI Staff Engineer): Engineering with AI

02

Vibe coding

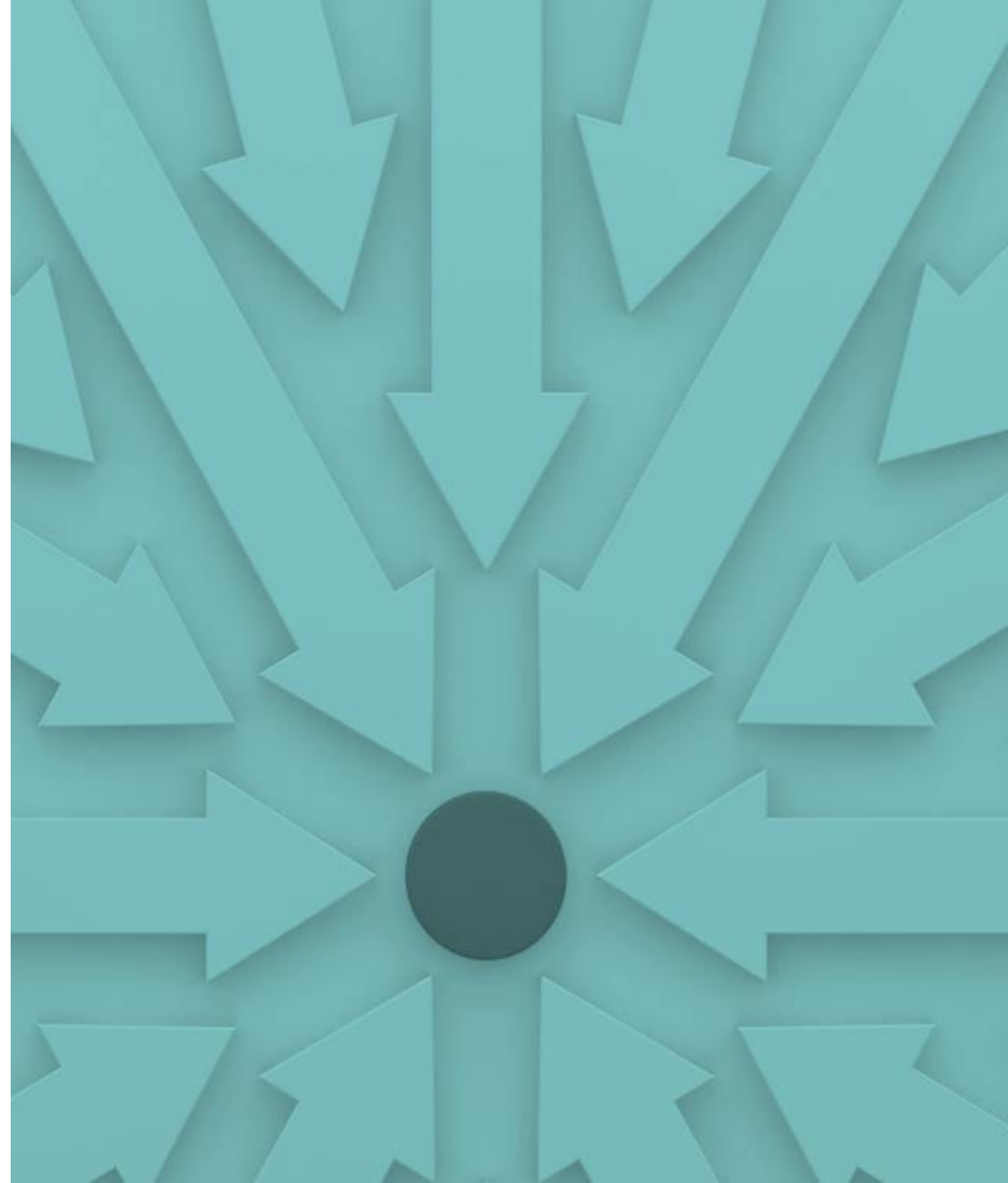
03

David Oort Alonso (CEO): Introduction to Bloom

04

Build & Experiment

The paradigm shift:
moving from
syntax-first to
intent-first.



Vibe Coding

Vibe coding refers to a conversational mode of software development in which the user does not interact with the code directly, but steers an AI coding assistant through natural-language instructions, feedback, and testing.

The development process becomes iterative: the user specifies intent, the AI implements it in code, the user evaluates the result, and further prompts are used to correct, extend, or refine the application.



Andrej Karpathy ✓

@karpathy



There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

12:17 AM · Feb 3, 2025 · **4.1M** Views

Development Flows

Traditional coding: You design, write, debug and maintain the code yourself, using conventional development tools.

AI-assisted coding: AI suggests code, explains concepts, completes functions or helps debug, but you remain fully in control of implementation.

Agentic coding: You delegate multi-step development tasks to an AI agent that can navigate the codebase, modify files, execute commands, run tests and iterate autonomously. You define objectives, constraints and acceptance criteria, then supervise and review the results.

Vibe coding: You specify intent in natural language, let AI handle most of the implementation, and judge success mainly from observed behaviour and output, often without closely inspecting or understanding the underlying code.

Spec-driven development with AI: You define detailed requirements, interfaces, architecture and tests before delegating implementation to AI. AI does much of the coding, but correctness is assessed against explicit specifications rather than intuition.

Vibe Coding Stack

1. Client Layer (The Interface)

Device

Native iOS / Android app

Role

Captures "user intent" (voice / text / gestures)

Tech

React Native shell that interprets the "vibe"

Intent



2. Orchestration Layer (The Brain)

AI Agent

The orchestrator; plans the app structure

Code Engine

Generates React Native code on the fly

Logic Mapping

Connects UI components to backend actions

Plan + Code



3. Infrastructure Layer (The Cloud)

Cloud Database

Serverless dynamic database; schema evolves with features

API

Hidden GraphQL / REST layer handling data flow

Deployment

App Clips + OTA updates push app to phone

Data + Deploy

Bloom

What is Bloom?

<https://bloom.diy/>

Bloom is for rapid prototyping of small, useful apps.

It is strongest when you want to test an idea fast, validate a workflow, or build an MVP without getting stuck in frontend, backend, and deployment details. That framing is also consistent with Bloom's own messaging around going from idea to production-ready apps quickly.



Startupticker.ch

20,422 followers

8mo • 🇨🇭

+ Follow

Founded this year, [Bloom \(YC X25\)](#) has already drawn tens of thousands of users to its no-code platform, which builds both frontend and backend in under two minutes. It delivers production-ready mobile apps developers can share.

The Swiss-US startup founded by [David Oort Alonso](#) and [Sirian Maathuis](#) has recently secured \$3.4M in pre-seed funding led by [Moonfire](#) with participation from Y Combinator and Pioneer Fund to fast-track growth through hiring.

Learn more about the story behind the headlines in our article: <https://lnkd.in/emTNGAKW>



Plan before Build

Start with the **problem**. A good Bloom use case is usually: **small, concrete, repetitive, and valuable**.

Use AI to draft, critique and refine a clear implementation plan before starting development.

- Start with **one specific task**
- Specify the **target user**
- Define the **format of the answer**
- Add **constraints** like length, tone, and what not to do
- Think of a **normal input** and a **messy input**
- Revise the prompt before building
- Structure for AI

Bloom offers a free tier with 2000 credits to the students, enough for [basic experimentation](#) and small prototypes. You may purchase extra credits if needed.

Clear and Simple Workflow

Clear input → transformation → output

Bloom-style apps are easiest when the workflow is simple:

- user enters or uploads something
- app processes it
- app returns something useful

Examples:

- input: messy notes → output: structured summary
- input: task list → output: prioritized plan
- input: symptom log → output: clean timeline
- input: study topic → output: quiz questions

Use Case Assessment

The 5-question filter

A use case is good if they can answer “yes” to most of these:

1. Can I explain the app in one sentence?
2. Does it solve one specific problem?
3. Would someone use it?
4. Can I demo value in under 30 seconds?
5. Can I build a first useful version in this session?

Examples that pass:

- “An app that turns a lecture topic into three oral exam questions.”
- “An app that helps lab students log sample preparation steps.”
- “An app that converts raw meeting notes into action items.”

‘Berufe entdecken’ Application

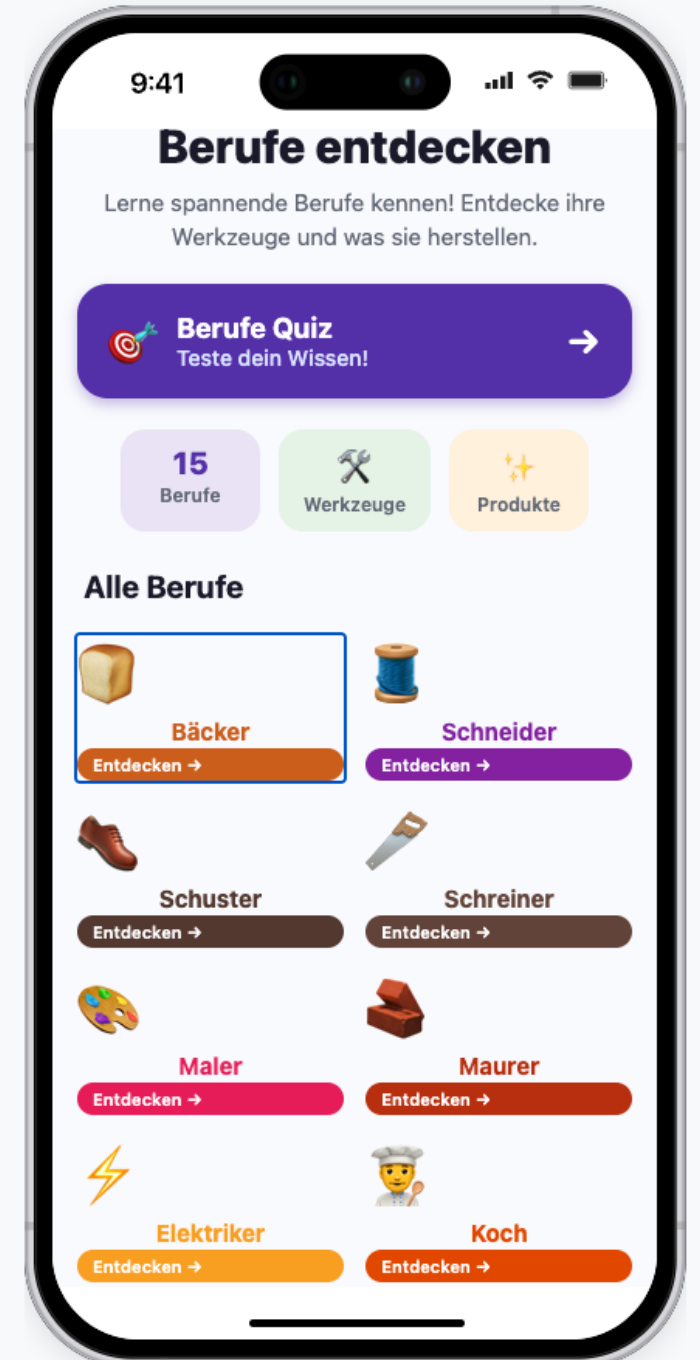
User: primary-school student preparing for an NMG test.

Problem: fragmented learning material, limited parental time, low motivation for paper-based exercises.

Idea: mobile quiz and study helper.

How this could be extended:

- Not only for a single NMG test, but for **all NMG topics**
- Share with other parents
- Enable other parents to create content and share the workload
- Build a community around it



Prompt Template

You are a helpful assistant for [target user].

Your task is to [specific task].

Context:

- The user is using this app for [situation].
- The goal is [desired outcome].

Instructions:

- Be accurate and concise.
- Ask for missing information only if essential.
- Do not invent facts.
- Keep the output practical and easy to use.

Output format:

1. [section one]
2. [section two]
3. [section three]

User input:

{{user_input}}

Agenda

01

Bartosz Nadworny (AI Staff Engineer): Engineering with AI

02

Vibe coding

03

David Oort Alonso (CEO): Introduction to Bloom

04

Build & Experiment

Agenda

01

Bartosz Nadworny (AI Staff Engineer): Engineering with AI

02

Vibe coding

03

David Oort Alonso (CEO): Introduction to Bloom

04

Build & Experiment



Build & Experiment

Build a prototype of your product idea, test your idea, and if time/credit allows, iterate on it once or twice.

Errors, bugs, unexpected behaviour, and the occasional “why on earth is it doing that?” moment are all part of the process. The goal is not to produce something perfect, but to learn by building, testing, and improving.

Present what they have built to the class in pitch form.

You are also welcome to use other development tools, such as **Claude Code, Cursor, or similar tools**, if you prefer.

Please note that **course-provided credits are only available for Bloom**; we cannot provide credits or subscriptions for other tools.

Final Submission

The deadline for the reflection form is **10 October 2026 for all students**

What problem are you trying to solve, and for whom? Briefly describe the user and the problem.

What did you actually build? Briefly describe what your prototype can do at the end of the session.

What was the main challenge you encountered while building the prototype? This can be a technical challenge, a product or design decision, prompting, debugging, scope, or something else.

What is one thing you learned from building the prototype that you did not expect beforehand?

What did you like about Bloom, or the alternative tool you used? What made building easier, faster, or more effective?

What did you miss, find difficult, or wish the tool did better?

If you had another few hours, what would you change or build next?

Video recording link (only if you did not present in class on 3 October)

Please provide a link to a 2–3 minute video presenting your prototype. Important: Make sure the video is shared so that it can be opened directly from the link without requesting access.